HACKATHON GIVAUDAN · HETIC · ÉQUIPE 7

Benchmark **LLM locaux**

Quels modèles Ollama, exécutables sur un PC de 16 Go, pour un assistant qui aide les commerciaux de Givaudan à anticiper les besoins de leurs clients ?

Version 0.1 · préliminaire · 10 novembre 2026

PRODUCT
Nathan · Maud

DATA
Judicha · Abdourahmane · Fidélia

TECH
Maxime

01 Contexte et objectif

Choisir, avec peu de temps, une architecture et des modèles qui tiennent sur nos ordinateurs et répondent juste.

La mission

Nous construisons une application qui prépare les commerciaux de Givaudan aux besoins de leurs clients. Elle analyse des **sources externes** (réglementation, veille marché) et les **données internes** (clients, produits, formules). Exemple type : lorsqu'une nouvelle restriction sur un ingrédient est publiée, l'application identifie les clients concernés et aide le commercial à les contacter avec une proposition de reformulation.

Les contraintes

MÉMOIRE

16 Go RAM

≈ 8 à 10 Go réellement disponibles pour le modèle, une fois Windows et les outils de développement ouverts.

EXÉCUTION

100 % local

Modèles servis par Ollama sur le poste. Les données clients ne quittent pas la machine.

TEMPS

Hackathon

Protocole court, automatisé, rejouable en une commande.

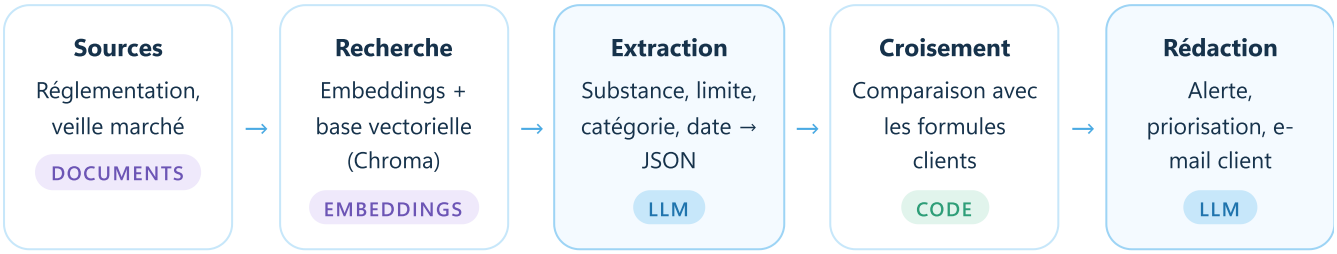
Ce que le benchmark doit trancher

- **Quel LLM** pour extraire l'information des sources et rédiger pour le commercial ?
- **Quel modèle d'embeddings** pour retrouver le bon passage, avec des questions en français et des sources souvent en anglais ?
- **Quelle répartition des rôles** entre le LLM et le code applicatif ?

> **Règle de décision.** Nous retiendrons le **plus petit modèle** qui atteint le niveau de qualité visé (≥ 80 % à la notation, aucune information inventée sur les questions sans réponse) avec une latence acceptable en démonstration (≈ 10 s par réponse).

02 Architecture envisagée

Un RAG (recherche documentaire + génération), où chaque brique fait ce qu'elle fait le mieux.



Un prototype fonctionnel de la partie recherche + génération existe déjà (API FastAPI, Chroma, Ollama).

Pourquoi le croisement est confié au code

Comparer une concentration à un seuil, filtrer des produits par catégorie ou faire un calcul de dilution sont des opérations exactes. Les premiers tests (section 05) montrent que les petits modèles s'y trompent régulièrement (« 0,9 % > 1,2 % »). Le LLM est donc limité à ce qu'il fait bien : **comprendre un texte** et **rédiger**. Le code garantit l'exactitude des chiffres présentés au commercial.

Conséquences sur le choix des modèles

- **Extraction structurée (JSON) fiable** : critère n°1 pour le LLM.
- **Fidélité aux sources** : dire « non mentionné » plutôt qu'inventer une limite réglementaire.
- **Multilingue** : questions en français, textes réglementaires souvent en anglais. Cela vaut pour le LLM comme pour les embeddings.
- **Taille** : modèles de 8 milliards de paramètres maximum en quantification 4 bits (≈ 5 Go). Au-delà, la machine bascule sur le disque et devient inutilisable.

03 Modèles sélectionnés et avis

Trois LLM de tailles croissantes et deux modèles d'embeddings, tous installés via Ollama.

qwen2.5:7b

Candidat principal · à tester

LLM d'extraction et de rédaction — Alibaba Qwen 2.5, 7 milliards de paramètres

Téléchargement **4,7 Go** | RAM estimée **5–6 Go** | Vitesse estimée **3–6 tok/s** (CPU)

ATOUTS ATTENDUS

- Réputé solide en sortie JSON et en suivi d'instructions
- Bon niveau en français et en anglais
- Contexte long, adapté aux extraits de textes réglementaires

RISQUES

- Latence élevée sans GPU (20–40 s par réponse estimées)
- Occupe plus de la moitié de la mémoire disponible

Avis : le meilleur rapport qualité/taille attendu dans notre budget mémoire. C'est le test prioritaire de la version 0.2. Si la latence est trop forte, on le réservera à l'extraction (faite une seule fois par texte) et on gardera un modèle plus léger pour la rédaction.

llama3.2:3b

Compromis · testé

LLM léger — Meta Llama 3.2, 3 milliards de paramètres

Téléchargement **2,0 Go** | RAM mesurée **2,9 Go** | Vitesse mesurée **9,5 tok/s** | Note **53 %**

POINTS FORTS OBSERVÉS

- Refuse correctement quand l'information est absente
- N'invente pas de limite sur la coumarine
- Bonne comparaison de seuil sur une question isolée

POINTS FAIBLES OBSERVÉS

- Erreur grave : déclare interdit un ingrédient autorisé dans les shampooings
- Croisements multiples et priorisation incohérents
- Lent sur les réponses longues (jusqu'à 49 s)

Avis : le plus fidèle aux sources des deux modèles testés, mais pas assez fiable pour être utilisé seul. Solution de repli si le 7B est trop lent.

qwen2.5:1.5b

Référence basse · testé

LLM très léger — modèle du prototype initial

Téléchargement **1,0 Go** | RAM mesurée **1,2 Go** | Vitesse mesurée **18,6 tok/s** | Note **36 %**

POINTS FORTS OBSERVÉS

- Très rapide (≈ 8 s par réponse)
- JSON syntaxiquement valide
- Bon résumé en français

POINTS FAIBLES OBSERVÉS

- Invente des valeurs absentes des sources (limonène, coumarine)
- Confond les catégories de produits
- Déclare non conformes tous les produits clients

Avis : écarté pour la production : les inventions sont incompatibles avec un usage réglementaire. Utile pour développer l'interface rapidement.

embeddinggemma

À tester

Embeddings — Google, ≈ 300 M de paramètres

621 Mo | **100+** langues | 768 dimensions

Multilingue : une question en français doit retrouver un passage en anglais. Léger, rapide sur CPU. Texte uniquement : **ne traite pas les images**. Nécessite des préfixes de tâche (requête / document) pour donner son meilleur niveau.

Avis : favori pour la recherche documentaire, à confirmer face au modèle actuel.

nomic-embed-text

En place

Embeddings — Nomic AI, ≈ 137 M de paramètres

274 Mo | 768 dimensions | contexte long

Modèle du prototype actuel. Très léger et éprouvé pour la recherche en anglais. Entraîné principalement sur de l'anglais : risque de moins bonne correspondance entre une question en français et une source en anglais.

Avis : base de comparaison. Conservé si l'écart avec EmbeddingGemma est faible.

04 Protocole

Automatisé, rejouable en une commande, centré sur les tâches réelles de l'application.

Jeu de test

12 questions métier, posées avec leurs sources dans le prompt. Les sources sont **volontairement fictives** (avis réglementaire en anglais, extrait de base clients, note de veille marché) : le modèle ne peut pas répondre de mémoire, on mesure donc sa capacité à **lire et respecter les sources**.

CATÉGORIE	QUESTIONS	CE QUI EST MESURÉ
Extraction	2	Retrouver une limite, une date, une catégorie de produit
Extraction JSON	1	Sortie structurée exploitable par le code
Croisement	2	Confronter la réglementation aux produits clients
Synthèse commerciale	2	E-mail client, priorisation des contacts
Refus	2	Dire « non mentionné » au lieu d'inventer
Fidélité au contexte	1	Ne pas transformer un seuil d'étiquetage en limite
Calcul	1	Dilution d'un ingrédient dans le produit fini
Résumé multilingue	1	Résumer en français un texte anglais

Conditions d'exécution

- Température 0, graine fixe (42) : résultats reproductibles
- Fenêtre de contexte fixée à 4 096 tokens, réponse limitée à 600 tokens
- Même prompt système pour tous les modèles
- Premier appel de chauffe exclu des mesures
- Un seul modèle chargé à la fois
- Même machine (16 Go), autres applications fermées

Mesures

AUTOMATIQUES

Mots-clés attendus, validité du JSON, détection du refus, langue de réponse. Latence totale, délai avant premier token, tokens par seconde, mémoire occupée (API Ollama).

NOTATION PAR IA (CLAUDE)

Chaque réponse est notée de 0 à 2 sur 4 critères : **exactitude**, **fidélité aux sources**, **complétude**, **utilité pour un commercial**. Total sur 8 par réponse, sur 96 par modèle.

Les contrôles automatiques seuls sont trompeurs (score maximal obtenu sur une réponse entièrement fausse) : la notation par IA est la mesure de référence. Une relecture humaine par l'équipe Data est prévue pour la version finale.

05 Premiers résultats

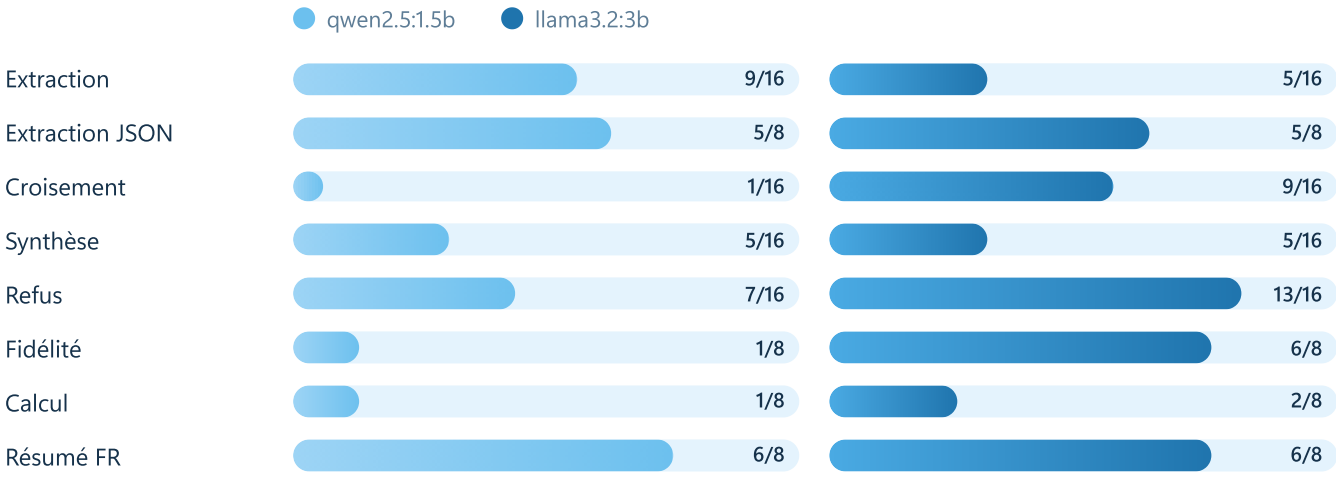
Exécution du 10 novembre 2026 sur les deux modèles légers. qwen2.5:7b et la comparaison des embeddings suivront.

LLAMA3.2:3B
53 % 51 / 96
17,5 s par réponse en moyenne

QWEN2.5:1.5B
36 % 35 / 96
8,1 s par réponse en moyenne

OBJECTIF
≥ 80 %
Aucun modèle testé ne l'atteint encore

Note par catégorie (notation IA)



Performances mesurées

MODÈLE	TOKENS / S	LATENCE MOYENNE	LATENCE MAX	RAM
`qwen2.5:1.5b`	18,6	8,1 s	23,7 s	1,2 Go
`llama3.2:3b`	9,5	17,5 s	49,4 s	2,9 Go

Enseignements

- **Aucun des deux modèles légers n'est utilisable seul** pour un outil réglementaire : trop d'erreurs factuelles sur des questions simples.
- **Croisement et calcul échouent chez les deux**. Cela confirme le choix d'architecture : la comparaison avec les données clients est faite par le code.
- **La taille joue surtout sur la fidélité** : le 3B invente nettement moins que le 1.5B. Le 7B devrait poursuivre cette tendance.

06 Recommandation provisoire

À confirmer par les tests de la version 0.2.

BRIQUE	CHOIX PROVISOIRE	ALTERNATIVE	STATUT
LLM extraction	`qwen2.5:7b`	`llama3.2:3b`	à tester
LLM rédaction	`qwen2.5:7b`	`llama3.2:3b` si latence trop forte	à tester
Embeddings	`embeddinggemma`	`nomic-embed-text`	à tester
Croisement clients	Code (Python / SQL)	—	validé
Développement de l'interface	`qwen2.5:1.5b`	—	validé

Prochaines étapes

VERSION 0.2

- Exécuter le benchmark sur `qwen2.5:7b`
- Comparer les embeddings (taux de bon passage retrouvé dans les 4 premiers résultats) sur des questions en français et des sources en anglais

VERSION FINALE

- Remplacer les sources fictives par les données du hackathon
- Relecture humaine des notes par l'équipe Data
- Mesure sur chaque poste de démonstration

Pistes écartées ou optionnelles

- **Modèles de plus de 8 milliards de paramètres** : écartés, ils dépassent la mémoire disponible.
- **Fine-tuning** : écarté, trop long pour le format hackathon, au profit du RAG.
- `bge-m3` (embeddings multilingues) : troisième candidat si EmbeddingGemma déçoit.
- `gemma3:4b` (modèle de vision) : uniquement si les sources contiennent des documents scannés ou des tableaux en image. Il les convertirait en texte avant l'indexation.

Reproduire le benchmark

```
ollama pull qwen2.5:1.5b ; ollama pull llama3.2:3b ; ollama pull qwen2.5:7b
ollama pull nomic-embed-text ; ollama pull embeddinggemma
python bench.py --models qwen2.5:1.5b llama3.2:3b qwen2.5:7b
```

Le script (Python standard, sans dépendance) produit un fichier de résultats détaillé et une feuille de réponses côte à côte pour la notation.